

Graphite

The whole project from zero: what the problem is, what we built, why, and where it stands.
Repo: github.com/rishitkam/graphite

1. The problem in plain words

A bank watches every card payment. Some payments look odd: a new phone, a different city, a strange amount at 3am. The bank's system raises an **alert** on those. A human fraud analyst then opens the case and has to decide: did the real cardholder make this payment, or did someone else?

To decide, the analyst looks around the payment. What does this card normally do? Has this phone been used by other people? Were those people victims of fraud before? Has the bank seen a case like this in the past, and how did it end? Then the analyst writes up a verdict, blocks the card or clears it, and if it is fraud, files a report for regulators (a **SAR**, suspicious activity report).

This is slow, repetitive work. The hackathon asks: can an AI agent do this investigation, using a **graph database** (TigerGraph) to look around the payment the way an analyst would?

2. What the hackathon asks for

- **20 exam cases**, named HHG-001 to HHG-020. Each is one alert. We must produce an answer file per case in a fixed JSON format: verdict, fraud probability, pattern, which transactions are affected, connected cards and devices, evidence, similar past cases, and a SAR if needed.
- **Three pipelines**, each a different way of letting the AI look things up:
 1. **RAG**: search old case notes by text similarity, hand the closest ones to the model.
 2. **GraphRAG**: pull a fixed set of facts out of the graph (card history, device, customer, similar cases) and hand them to the model.
 3. **Agentic GraphRAG**: give the model tools that query the graph and let it decide what to look at, step by step.
- **Same model for all three**. The organizers said they score the **improvement between pipelines**, not raw accuracy, plus architecture, token efficiency and engineering. A cheap model with a smart pipeline can beat an expensive model with a lazy one.
- **TigerGraph is mandatory**, including its official MCP server (the standard way AI tools talk to it).

3. The data

An anonymised card payments dataset loaded into TigerGraph:

Thing	Count	What it is
Transactions	590,742	Every payment, with amount, time, product, channel, region
Customers / Cards	13,553 / 14,317	Who owns what. Cards are named like C08623-K2
Device profiles	9,436	A fingerprint of the phone or browser used (not unique per person)
Billing regions	332	Rough location of the card's billing address
Email domains	59	gmail.com, anonymous.com and so on
Closed cases	5,565	Past investigations with outcome and analyst notes: the bank's memory

In the graph these are all **nodes**, joined by **edges**: card MADE transaction, transaction FROM_DEVICE device, customer OWNS card, case INVOLVES transaction. About 2.3 million edges in total. The point of a graph is that "who else used this phone, and did any of them get defrauded?" is a two-hop walk instead of a slow multi-table join.

4. Why a graph at all

Fraud is relational. One stolen card on its own looks almost normal. What gives it away is the **connections**: the same device showing up on eight strangers' cards in one week, all "new device", all through a proxy. A text search over notes cannot see that. A graph walk sees it immediately.

That is the thesis Graphite tests: the more the pipeline can use graph structure, the better it should do, with the same model.

5. How Graphite works

```
alert --> gather evidence --> model proposes --> policy engine decides --> write back
          (differs per tier)   (same prompt,   (same rules R1-R10,   (case node in
                              same JSON)      no model involved) graph, SAR)
```

5.1 Only the evidence differs between tiers

All three pipelines share the **same prompt, the same output format and the same rule engine**. The only thing that changes is how evidence is gathered. So any difference in results is caused by retrieval, which is exactly what the judges want measured. Most teams write three different prompts, which makes their comparison meaningless.

5.2 The model proposes, rules decide

The model never blocks a card or files a report itself. It outputs an assessment (probability, pattern, evidence). A plain Python **policy engine** with ten written rules (R1 to R10) turns that into actions: freeze card, escalate, clear, file SAR. This is how real banks keep AI auditable, and it means a hallucination cannot trigger an action on its own.

5.3 Situation memory

For every past case we build a small 15-number "fingerprint" of its graph situation: was the device new, was there a proxy, how many strangers share the device, how far from the usual region, and so on. These are stored as vectors inside TigerGraph. For a new alert we find the most similar past situations and see how they ended. This alone, with zero model calls, is a strong baseline.

5.4 Correcting the bank's own bias

The bank only investigated a legitimate payment when its own model flagged it, so 85% of past cases are fraud. Read naively, memory says "almost everything is fraud". We correct for this mathematically (compare a neighbourhood's fraud share against the 85% overall share). Before the fix, memory called 18 of 20 exam cases fraud; after, 9. The README says about half are legitimate.

5.5 Recurring charge check (disputes)

When a customer says "I never made this purchase", sometimes it is their own forgotten subscription. Past data has no legitimate disputes, so memory cannot recognise this. We added a check: has this card paid the same amount for the same product many times before? HHG-003 disputes \$49.00 on a card with 53 earlier \$49.00 charges. Policy rule R7 covers exactly this. We now run this check automatically on every dispute, because on the first run the agent skipped it despite being told to.

5.6 Finding fraud rings before anyone reports them

A standout extra: Graphite does not just wait for alerts. It builds links between customers who share a suspicious device (new to both, mostly through proxies) and runs TigerGraph's connected-components algorithm to find **rings**. In Nov to Dec it found 6 rings covering 61 customers, 5 of which contain no exam case at all. These become proactive cases PRO-002 to PRO-006; three were judged fraud with SARs (8, 5 and 3 cards).

5.7 Everything through MCP

Every graph query in every pipeline goes through TigerGraph's official MCP server. We verified results are byte-identical to calling the database directly.

6. Testing honestly

A fraud eval is easy to fake by accident. We found and closed these leaks, each of which inflated scores:

Leak	Why it cheated	Fix
Alert trigger type	Predicted the label 100% in history	Neutral trigger text in eval
Risk score	91% predictive on its own; the model learns nothing	Eval cases are score-matched: fraud and cleared cases with the same score (within 0.01)
Timing	Case open time differed by outcome	Investigate at flagged time + 6 hours for all
Future data	Memory could contain the answer	Every query takes an "as of" time; eval cases are excluded from memory
Tuning on test	Overfitting the reported numbers	Separate dev split for tuning; code frozen before the held-out run

The held-out set: 160 score-matched cases (80 fraud, 80 cleared) plus 8 rare-pattern cases. The dev set: 40 separate cases. Seventeen automated tests enforce the no-leak rules.

7. Where things stand

Exam (done)

All 20 answered by the agentic pipeline, validated against the required format, written into the graph with SARs. Result: 10 fraud, 9 legitimate, 1 uncertain (HHG-009). About 1.8k to 6k tokens per case.

Held-out results (final)

168 cases never used for tuning, same model (gpt-oss-120b) for all three, run once on frozen code.

	Memory only (no LLM)	RAG	GraphRAG	Agentic
Accuracy	85.7%	44.0%	73.8%	75.6%
AUC (0.5 = coin flip)	0.919	0.433	0.805	0.852
Brier (lower is better)	0.109	0.341	0.179	0.158
Fraud pattern named correctly	n/a	8%	44%	51%
Tokens / case	0	1,296	2,679	4,818

Each step up the ladder helps: GraphRAG is +29.8 points over RAG, the agent +31.5, and the agent also ranks, calibrates and names patterns better than GraphRAG. RAG is below a coin flip because matched risk scores remove the easy signal, and notes that read alike are not situations that are alike. The no-model memory row scores highest, which is the finding: the signal lives in the graph, and the model explains it and proposes actions.

Still to do

- Public demo site (static build is ready; publishing needs your go-ahead).
- Demo video, blog post and social post (you).
- Rotate the Groq keys after the hackathon.

8. Why this can win

1. **A fair comparison.** One prompt, one rule engine, one model; only retrieval changes. The improvement numbers mean something.
2. **An honest eval.** Score-matched, leak-audited, frozen. Most submissions will report numbers that the risk score alone would get.

- 3. **Calibrated verdicts.** We do not call everything fraud. The bias correction and R7 check produce a believable split.
- 4. **Proactive ring detection** using TigerGraph's own graph algorithms: finds fraud no alert pointed at.
- 5. **Token efficiency.** A few thousand tokens per case, with a zero-token memory baseline to show what the graph gives for free.
- 6. **Auditable actions.** The model proposes, deterministic policy decides.

9. Glossary

AUC	How well the probabilities rank fraud above legitimate. 1.0 perfect, 0.5 random.
Brier	Error of the probabilities themselves. Lower is better.
Recall / specificity	Share of fraud caught / share of legitimate correctly cleared.
GSQL	TigerGraph's query language. Our queries are installed in the database.
MCP	Model Context Protocol, a standard way for AI apps to call tools like a database.
Vector / kNN	Numbers describing a thing; kNN finds the nearest ones. Used for notes and situations.
SAR	Suspicious Activity Report, the regulator filing for confirmed fraud.
Token	A chunk of text the model reads or writes; what model usage is billed and scored in.